

dbt Training

 Working Effectively · Module 09 · 90 min

Jinja and Macros

Stop copying the same CAST expression into every model. Start writing reusable SQL logic — and know exactly when not to.

Module 08 Was Already Jinja

You've been using it — now you'll write it

Module 08 — variables in SQL

```
WHERE amount > {{ var('min_deal_amount', 0) }}
```

This IS Jinja. `{{ var(...) }}` is an expression that dbt evaluates at compile time and substitutes into your SQL.

Module 09 — write your own Jinja

Variables let you substitute **constants**.

Macros let you substitute **SQL patterns** — parameterised templates you define once and call everywhere.

Today: build reusable SQL patterns.

The Problem

You wrote this CAST in 12 different staging models

stg_hubspot_deals.sql

```
SELECT
  deal_id,
  CAST(amount AS DOUBLE) AS amount,
  CAST(close_date AS DATE) AS expected_close_date
FROM {{ source('hubspot', 'deals') }}
```

stg_hubspot_contacts.sql

```
SELECT
  contact_id,
  CAST(score AS INTEGER) AS contact_score,
  CAST(created_at AS DATE) AS created_date
FROM {{ source('hubspot', 'contacts') }}
```

The hidden cost

Source data from Bronze is messy. When `amount` contains `'N/A'`, `CAST` throws a runtime error and fails the entire model run. You need `TRY_CAST` — but now you have to fix it in 12 places. And next time someone adds a 13th model, they'll use `CAST` again because there's no convention to follow.

Jinja Syntax — The Three Delimiters

A macro uses all three. Know which does what.

Delimiter	Purpose	Renders output?	Example	Expression
<code>{{ }}</code>	Expression — evaluates and outputs a value	Yes	<code>{{ ref('fct_deal') }}</code> , <code>{{ my_macro(arg) }}</code>	<code>if is_incremental() %}</code> , <code>{% macro name(args) %}</code>
<code>{% }</code>	Statement — logic, control flow, definitions	No	<code>{% if is_incremental() %}</code> , <code>{% macro name(args) %}</code>	<code>{# #}</code>
<code>{# }</code>	Comment — ignored entirely	No	<code>{# TODO: add null guard #}</code>	

Wrong — breaks immediately

```
{{ macro safe_cast(col, type) }}
    TRY_CAST({{ col }} AS {{ type }})
{{ endmacro }}
```

dbt raises a parse error. `{{ }}` tries to output the value of `macro safe_cast(col, type)` — which is nothing. The macro is never defined.

Correct

```
{% macro safe_cast(col, type) %}
    TRY_CAST({{ col }} AS {{ type }})
{% endmacro %}
```

`{% }` defines the macro without emitting output. `{{ }}` inside the body outputs the argument values into SQL.

Built-in dbt Jinja Variables

Three variables available in every model and macro

`{{ this }}` current model's relation

Used as: `{% if is_incremental() %} WHERE updated_at > (SELECT MAX(updated_at) FROM {{ this }}) {% endif %}`

Compiles to: `dev_schema.fct_deal` (or prod equivalent)

`{{ target }}` active profile target

Used as: `{% if target.name == 'prod' %}` — full dataset in prod, 90-day filter in dev

Properties: `target.name` · `target.schema` · `target.type`

`{{ model }}` current model metadata

Used as: `{{ model.name }}` — useful in generic macros that need to know their caller

Compiles to: `fct_deal` (the file name, no extension)

Your First Macro: `safe_cast`

macros/safe_cast.sql — definition

```
{% macro safe_cast(
    column_name,
    target_type,
    fallback=None
) %}
{%- if fallback is not none -%}
    COALESCE(
        TRY_CAST({{ column_name }}
            AS {{ target_type }}),
        {{ fallback }}
    )
{%- else -%}
    TRY_CAST({{ column_name }}
        AS {{ target_type }})
{%- endif -%}
{% endmacro %}
```

stg_hubspot_deals.sql — call

```
SELECT
    deal_id,
    {{ safe_cast(
        'amount',
        'DOUBLE'
    ) }} AS amount,
    {{ safe_cast(
        'close_date',
        'DATE'
    ) }} AS expected_close_date,
    {{ safe_cast(
        'deal_score',
        'INTEGER',
        '0'
    ) }} AS deal_score
FROM {{ source('hubspot', 'deals') }}
```

target/compiled/... — SQL sent to DuckDB

```
SELECT
    deal_id,
    TRY_CAST(amount AS DOUBLE)
```

```

    AS amount,
    TRY_CAST(close_date AS DATE)
    AS expected_close_date,
    COALESCE(
        TRY_CAST(deal_score AS INTEGER),
        0
    )
    AS deal_score
FROM main.raw_deals

```

No Jinja in the output. DuckDB receives plain SQL.

Macro Anatomy

Every macro follows the same pattern

```sql {all|1|2-3|4-8|9|all} {% macro macro\_name(parameter\_1, parameter\_2='default\_value') %} {# Optional Jinja comment explaining what this outputs #} -- SQL text that uses the parameters -- Use {{ parameter\_1 }} to output a parameter's value into SQL -- Use {% if %} / {% else %} / {% endif %} for conditional output {{ parameter\_1 }} AND {{ parameter\_2 }} {% endmacro %}

```

</div>

<div class="mt-4 grid grid-cols-3 gap-3 text-sm">
 <div v-click class="bg-slate-50 border border-slate-200 rounded-lg p-3">
 <div class="font-semibold text-slate-700 mb-1">Name and parameters</div>
 <div class="text-slate-600">Parameters can have default values. Callers can omit optional parameters.
 </div>
</div>

<div v-click class="bg-slate-50 border border-slate-200 rounded-lg p-3">
 <div class="font-semibold text-slate-700 mb-1">The body</div>
 <div class="text-slate-600">Everything between <code>{% macro %}</code> and <code>{% endmacro %}</code>
 is the output template. Use <code>{{ }}</code> to insert parameter values.</div>
</div>

<div v-click class="bg-slate-50 border border-slate-200 rounded-lg p-3">
 <div class="font-semibold text-slate-700 mb-1">Calling the macro</div>
 <div class="text-slate-600"><code>{{ macro_name(arg1, arg2) }}</code> – expression delimiter. The
 compiled SQL replaces the call site.</div>
</div>
</div>

<!--
This slide makes the anatomy explicit and self-contained. Use the line highlights to walk through the code
block step by step.

Line 1: {% macro %} – statement, defines the name and parameter list.

```

Lines 2-3: default values – callers can omit the parameter and the default applies.

Lines 4-8: the body – this is what gets output when the macro is called. Mix of SQL and Jinja expressions.

Line 9: {% endmacro %} – statement, closes the definition.

Key takeaway: macros are just named Jinja templates. When you call `{{ safe_cast('amount', 'DOUBLE') }}`, dbt substitutes the arguments into the template and outputs the resulting SQL string. Nothing more.

Checkpoint: "If you call `{{ safe_cast('score', 'INTEGER') }}` and fallback is not provided, what does it output?" → `TRY_CAST(score AS INTEGER)`. The else branch of the macro.

-->

---

```
`dbt_utils.generate_surrogate_key`
```

### Why surrogate keys? Where to generate them?

```
<div class="mt-4 grid grid-cols-2 gap-6">
```

```
<div class="space-y-3">
```

```
 <div class="bg-white border border-slate-200 rounded-xl p-4 text-sm">
```

```
 <div class="font-semibold text-slate-700 mb-2">Why a surrogate key?</div>
```

```
 <div class="text-slate-600 space-y-1">
```

```
 <div v-click>Natural keys (HubSpot IDs) are strings that can change, merge, or be reused.</div>
```

```
 <div v-click>A hash key is stable, join-safe across systems, and works as a consistent dimension key in Power BI.</div>
```

```
 <div v-click>Every Silver model needs one. Without it, joins to fact tables are fragile.</div>
```

```
 </div>
```

```
 </div>
```

```
 <div v-click class="bg-amber-50 border border-amber-200 rounded-xl p-4 text-sm">
```

```
 <div class="font-semibold text-amber-800 mb-2">Where to call it</div>
```

```
 <div class="text-amber-900">In the source CTE of each model – not in a shared macro.
```

```
 Surrogate key generation is part of the row's identity. Put it where the row is first selected.</div>
```

```
 </div>
```

```
</div>
```

```
<div class="bg-white border border-slate-200 rounded-xl p-4">
```

```
 <div class="text-xs font-mono text-slate-400 mb-2">models/silver/fct_deal.sql</div>
```

```
```sql
```

```
WITH source AS (
```

```
  SELECT
```

```
    {{ dbt_utils.generate_surrogate_key(
```

```
      ['deal_id']
```

```
    ) }}      AS deal_key,
```

```
    deal_id,
```

```

        deal_name,
        pipeline_id,
        amount,
        expected_close_date,
        ingested_at
    FROM {{ ref('stg_hubspot__deals') }}
)

SELECT * FROM source

```

Compiles to: `MD5(CAST(deal_id AS TEXT)) AS deal_key`

The Macro Antipattern

What happens when business logic moves into Jinja

macros/classify_deal.sql — do not write this

```

{% macro classify_deal(amount_col, stage_col) %}
CASE
    WHEN {{ stage_col }} = 'closed-won'
        AND {{ amount_col }} > 50000
    THEN 'enterprise-closed'
    WHEN {{ stage_col }} = 'closed-won'
        AND {{ amount_col }} > 10000
    THEN 'mid-market-closed'
    WHEN {{ stage_col }} IN ('proposal','demo')
        AND {{ amount_col }} > 50000
    THEN 'enterprise-pipeline'
    -- ... 20 more lines of business rules
END
{% endmacro %}

```

- ✗ Business rules are invisible to SQL analysts
- ✗ Cannot write an `accepted_values` test targeting this logic
- ✗ A BI developer reading `fct_deal` cannot understand what `deal_segment` means without reading the macro file `models/silver/fct_deal.sql` — write this instead

```

WITH classified AS (
    SELECT
        deal_id,
        amount,
        pipeline_stage,

```

```

CASE
  WHEN pipeline_stage = 'closed-won'
    AND amount > 50000
    THEN 'enterprise-closed'
  WHEN pipeline_stage = 'closed-won'
    AND amount > 10000
    THEN 'mid-market-closed'
  WHEN pipeline_stage IN ('proposal', 'demo')
    AND amount > 50000
    THEN 'enterprise-pipeline'
  ELSE 'other'
END AS deal_segment
FROM {{ ref('stg_hubspot__deals') }}
)

```

- ✓ Business rules visible in lineage and PR review
- ✓ `accepted_values` test targets `fct_deal.deal_segment` directly
- ✓ Any SQL analyst can read the model and understand the segmentation

When to Use What

Decision table: macro vs. CTE vs. ephemeral

| Situation | Right tool | Reason | |---|---|---| | Same `TRY\_CAST` pattern in 8+ staging models | Macro | Structural — purely mechanical, no business meaning | | Surrogate key hash over a column | `dbt\_utils.generate\_surrogate\_key` | Already a package macro — call it directly | | `CASE WHEN` mapping pipeline stages to categories | SQL CTE in the model | Business logic — visible, reviewable, testable | | Intermediate join used by exactly one model | SQL CTE or ephemeral | Single use — abstraction adds complexity, no benefit | | `CASE WHEN` that classifies deal size across 15 models | Macro (with docstring) | Repeated business rule — document it thoroughly | | 50-line Jinja template encoding revenue allocation rules | Never | Jinja is not a good language for business logic |

4-question checklist before writing a macro

1. Is this pattern **structural** (not business logic)? If no — use a CTE.
2. Does it repeat across **3+ models**? If no — use a CTE.
3. Would a **CTE work instead**? If yes — prefer the CTE.
4. Is it **testable on its own**? If no — reconsider whether Jinja is the right tool.

Hooks

Run SQL before or after a model executes

`dbt_project.yml` — project-level hooks

```
models:
  analytics:
    silver:
      facts:
        +post-hook:
          - >
            ALTER TABLE {{ this }}
            ADD PRIMARY KEY (deal_key)
            NOT ENFORCED
```

model config — inline hook

```
{{ config(
  materialized = 'table',
  post_hook    = [
    "GRANT SELECT ON {{ this }}"
    TO ROLE REPORTER"
  ]
) }}
```

pre-hook

Runs before the model SQL. Use for: drop temp tables, set session variables, write an audit log entry.

post-hook

Runs after the model SQL. Use for: add constraints, grant permissions, write a completion audit log entry.

Hooks are powerful and hard to debug. Prefer solving the problem in dbt config before reaching for hooks.

Packages

Install pre-built macro libraries with `packages.yml`

packages.yml (project root)

```
packages:
- package: dbt-labs/dbt_utils
  version: 1.3.0
- package: calogica/dbt_expectations
  version: 0.10.4
```

After adding or updating `packages.yml`, run: `dbt deps` Downloads to `dbt_packages/` — gitignored. Run `dbt deps` after every clone.

dbt_utils macros used in this project

`generate_surrogate_key([cols])` — MD5 hash surrogate key

`star(from=ref('model'), except=[...])` — SELECT * except columns

`pivot(col, values, ...)` — pivot a column into boolean columns

`dbt_expectations` tests used in this project

`expect_column_values_to_be_between` — numeric range check

`expect_column_pair_values_to_be_equal` — assert two columns match

layout: default

background: '#f9f8f5'

Exercise — 25 min

Scenario: You are adding a safe type coercion convention and surrogate key generation to the exercise project. All three tasks must pass `dbt build`.

Task 1 — Write the macro

`macros/safe_cast.sql`

Create `macros/safe_cast.sql`. Parameters: `column_name`, `target_type`, `fallback=None`. Without fallback:

`TRY_CAST(column_name AS target_type)`. With fallback: `COALESCE(TRY_CAST(...), fallback)`. Verify with `dbt compile --select stg_hubspot_deals`.

Task 2 — Use the macro

`stg_hubspot_deals.sql`

Replace the raw `amount` column with `{{ safe_cast('amount', 'DOUBLE') }} AS amount`. Run `dbt build --select stg_hubspot_deals`. Check compiled output confirms `TRY_CAST(amount AS DOUBLE)`.

Task 3 — Surrogate key

`fct_deal.sql`

Add `{{ dbt_utils.generate_surrogate_key(['deal_id']) }} AS deal_key` as the first column in the source CTE of `fct_deal.sql`. Run `dbt build --select fct_deal`. Confirm `deal_key` compiles to an MD5 expression.

Bonus

Open any two Silver models. Find one expression that repeats across them. Is it a good macro candidate? Write the macro signature if yes. Explain why not if no. Use the decision table from the previous slide.

layout: center

Key Takeaways

Macros compile to plain SQL. Jinja is an authoring tool — DuckDB and Snowflake never see it. When in doubt, run `dbt compile` and read the output.

Use macros for structural patterns, not business logic. Type coercion and key generation belong in macros. CASE WHEN logic that classifies deals belongs in SQL — where it's visible, testable, and reviewable.

Call `dbt deps` after every clone. `dbt_packages/` is gitignored. If `generate_surrogate_key` isn't found, you haven't run `dbt deps`.

layout: center

Module 09 Complete

Next: Module 10

Snapshots and Slowly Changing Dimensions (SCD2)

Prep Q1: What is a slowly changing dimension? What type does dbt snapshots implement by default?

Prep Q2: If a contact's email changes in HubSpot, how would you preserve both the old and new value with history?

Prep Q3: What columns does a dbt snapshot add to track row history?

Prep Q4: In `dim_contact`, `contact_key` is a hash of `hubspot_contact_id`. If a contact's email changes, does `contact_key` change? Why does that matter for snapshot strategy?